Rezolvare sistem superior Δ (SST)

$$x_i = \frac{b_i - \sum_{j=i+1}^{n} a_{ij} \cdot x_j}{a_{ii}}$$

$i = n : -1 : 1$

complexitate : $O(n^2)$

Scanned with CamScanner

```
function x = SST(U, b)
    n = length(b);
    x = zeros(n, 1);
    for i = n:-1:1
        x(i) = (b(i) - U(i, i+1:n)*
                * x(i+1:n))/
                / U(i, i);
    endfor
endfunction
```

Scanned with CamScanner

Rezolvare sistem inferior Δ (SIT)

$$x_i = \frac{b_i - \sum_{j=1}^{i-1} l_{ij} x_j}{l_{ii}},$$

$$i = 1 : n$$

complexitate: $O(n^2)$

Scanned with CamScanner

```
function x = SIT(L, b)
    n = length(b);
    x = zeros(n, 1);
    for i = 1 : n
        x(i) = (b(i) - L(i, 1:i-1)*
            * x(1:i-1)) /
            / L(i, i);
    endfor
endfunction
```

Scanned with CamScanner

Crout:

$u_{ii} = 1$, $i = 1:n$

$p = 1:n$

$i = p:n$

$$l_{ip} = a_{ip} - \sum_{k=1}^{p-1} l_{ik} \cdot u_{kp}$$

$j = (p+1):n$

$$u_{pj} = \frac{a_{pj} - \sum_{k=1}^{p-1} l_{pk} \cdot u_{kj}}{l_{pp}}$$

complexitate: $O(n^3)$

condiție: toți minorii principali $\neq 0$

- instabil

```
function [L U x] = crout(A, b)
    [n n] = size(A);
    L = zeros(n);
    U = eye(n);
    for p = 1 : n
      for i = p : n
        L(i, p) = A(i, p) - L(i, 1:(p-1)) *
                  * U(1:(p-1), p);
      endfor
      for j = (p+1) : n
        U(p, j) = (A(p, j) - L(p, 1:(p-1)) *
                  * U(1:(p-1), j)) /
                  / L(p, p);
      endfor
    endfor
    y = SIT(L, b);
    x = SST(U, y);
endfunction
```

Doolittle:

$l_{ii} = 1, \quad i = 1:n$

$p = 1:n$

$j = p:n$

$$u_{pj} = a_{pj} - \sum_{k=1}^{p-1} l_{pk} \cdot u_{kj}$$

$i = p+1:n$

$$l_{ip} = \frac{a_{ip} - \sum_{k=1}^{p-1} l_{ik} \cdot u_{kp}}{u_{pp}}$$

complexitate: $O(n^3)$

condiție: toți minorii principali $\neq 0$

- instabil

Scanned with CamScanner

```
function [L U x] = doolittle (A, b)
    [n n] = size (A);
    L = eye (n);
    U = zeros (n);
    for p = 1 : n
       for j = p : n
          U(p, j) = A(p, j) - L(p, 1 : (p-1)) *
                    * U(1 : (p-1), j);
       endfor
       for i = (p+1) : n
          L(i, p) = (A(i, p) - L(i, 1 : (p-1)) *
                    * U(1 : (p-1), p)) /
                    / U(p, p);
       endfor
    endfor
    y = SFT (L, b);
    x = SST (U, y);
endfunction
```

Scanned with CamScanner

Cholesky

$U = L^T$

$j = 1 : n$

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}$$

~~$l_{jj} = a_{jj}$~~

$i = (j+1) : n$

$$l_{ij} = \frac{a_{ij} - \sum_{k=1}^{j-1} l_{jk} \cdot l_{ik}}{l_{jj}}$$

(vectorizăm doar sumele)

complexitate : $O(n^3)$

condiție : → matrice simetrică, pozitiv definită

↓ sau valori proprii pozitive

↓ sau diag dominantă

- stabil ≃

($A = A^T$ și $x^T A x > 0, \forall x \in \mathbb{R}^n, x \neq 0$)

Scanned with CamScanner

```
function [L U x] = cholesky (A, b)
    [u n] = size(A);
    L = zeros(u);
    U = zeros(n);
    for j = 1 : n
        L(j,j) = sqrt( A(j,j) - L(j,1:(j-1)) * L(j,1:(j-1))' )
        for i = (j+1) : n
            L(i,j) = (A(i,j) - L(j,1:(j-1)) * L(i,1:(j-1))') / L(j,j);
        endfor
    endfor
    U = L';
    y = SIT (L,b);
    x = SST (U,y);
endfunction
```

transpose

Gram-Schmidt:

Q - ortog; R - sup. Δ

$j = 1 : n$

$k = 1 : j-1$

$$r_{kj} = q_k^T \cdot a_j$$

$$aux = a_j - \sum_{k=1}^{j-1} r_{kj} \cdot q_k$$

(v. de la ALGATEN)

$$r_{jj} = norm(aux);$$

$$q_j = \frac{aux}{r_{jj}}$$

complexitate: $O(n^3)$

→ instabil

Scanned with CamScanner

```
function [Q R x] = gram_schmidt(A,b)
    [m n] = size(A);
    Q = zeros(m,n);
    R = zeros(n);
    for j = 1:n
        for i = 1:j-1
            R(i,j) = Q(:,i)' * A(:,j);
        endfor
        aux = A(:,j) - Q(:,1:j-1) *
                        * R(1:j-1,j);
        R(j,j) = norm(aux);
        Q(:,j) = aux / R(j,j);
    endfor
    x = SST(R, Q' * b);
endfunction
```

Scanned with CamScanner

Gram-Schmidt modificat

$i = 1 : n$

$$r_{ii} = \| a_i \|_2$$

$$q_i = \frac{a_i}{r_{ii}}$$

$j = i+1 : n$

$$r_{ij} = q_i^T \cdot a_j$$

$$a_j = a_j - q_i r_{ij}$$

→ stabil !!!

```
function [Q R x] = gram_schmidt_mod(A, b)
    [n n] = size(A);
    R = zeros(n);
    Q = zeros(n);
    for i = 1:n
        R(i,i) = norm(A(:,i));
        Q(:,i) = A(:,i) / R(i,i);
        for j = i + 1:n
            R(i,j) = Q(:,i)' * A(:,j);
            A(:,j) = A(:,j) - Q(:,i) *
                        * R(i,j);
        endfor
    endfor
    x = SST(R, Q' * b);
endfunction
```

Scanned with CamScanner

Householder

→ la un pas „zeroizează" toată coloana sub diagonala principală

$$\sigma_p = semn(a_{pp}) \sqrt{\sum_{k=p}^{m} a_{kp}^2}$$

$$v_p = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ a_{pp} + \sigma_p \\ a_{p,p+1} \\ \vdots \\ a_{np} \end{bmatrix}$$

$$v_{pp} = a_{pp} + \sigma_p$$

$$v_{ip} = a_{ip}, \; i = p+1 : m$$

$$H_p = I_m - \frac{2 v_p \cdot v_p^T}{v_p^T \cdot v_p}, \quad H_p \in \mathbb{R}^{m \times m}$$

$$H = H_{min(m-1,n)} \cdot \ldots \cdot H_p \cdot \ldots \cdot H_1 \cdot I_m$$

$$R = HA$$

$$Q = H^T$$

$$b_{final} = Hb$$

$b_1 = b$
$b_2 = H_1 b_1$
$b_3 = H_2 b_2$
$\ldots$
$b_{final} = H_{min(m-1,n)} \ldots b_{min(m-1,n)}$

complexitate: $O(n^3)$

→ STABIL

Scanned with CamScanner

```
function [Q R x] = householder(A,b)
    [m n] = size(A);
    Q = eye(m);
    for p = 1 : min(m-1, n)
        vp = zeros(m, 1);
        sigma = norm(A(p:m, p));
        if (A(p,p) < 0)
            sigma = - sigma;
        endif
        vp(p) = A(p,p) + sigma;
        vp(p+1:m) = A(p+1:m, p);
        Hp = eye(m,m) - 2*(vp * vp')/
                        /(vp' * vp);
        A = Hp * A;
        b = Hp * b;
        Q = Hp * Q;
    endfor
    R = A;
    Q = Q';
    x = SST(R,b);    endfunction
```

Scanned with CamScanner

Givens

→ zerorizează un singur element la un pas

$$G_{lk} = \begin{bmatrix} 1 & & & & 0 \\ & \ddots & & & \\ & & c & \cdots & s \\ & & \vdots & & \vdots \\ & & s & \cdots & c \\ & & & & & \ddots \\ 0 & & & & & & 1 \end{bmatrix} \begin{matrix} \\ \\ \leftarrow k \\ \\ \leftarrow l \\ \\ \end{matrix}$$

$$\begin{matrix} & & \downarrow & & \downarrow \\ & & k & & l \end{matrix}$$

→ modifică doar liniile k și l !!

$$c = \frac{a_{kk}}{\sqrt{a_{kk}^2 + a_{lk}^2}}$$

$$s = \frac{-a_{lk}}{\sqrt{a_{kk}^2 + a_{lk}^2}}$$

[redacted] ca să zerorizezi a_{lk}:

$$A = G_{lk} \cdot A$$

Scanned with CamScanner

```
function [Q R x] = givens (A, b)
    [m n] = size(A);
    Q = eye(m);
    for k = 1 : min(m-1, n)
        for l = k+1 : m
            r = sqrt(A(k,k)^2 + A(l,k)^2);
            c = A(k,k) / r;
            s = -A(l,k) / r;
            % A_{p+1} = G_kl * A_p :
            aux = [c -s; s c] * [A(k,k:n);
                                 A(l,k:n)];
            A(k,k:n) = aux(1,:);
            A(l,k:n) = aux(2,:);
            % b_{p+1} = G_kl * b_p :
            aux = [c -s; s c] * [b(k); b(l)];
            b(k) = aux(1);
            b(l) = aux(2);
            % Q_{p+1} = G_kl * Q_p :
            aux = [c -s; s c] * [Q(k,1:n);
                                 Q(l,1:n)];
            Q(k,1:n) = aux(1,:);
            Q(l,1:n) = aux(2,:);
        endfor
    endfor
    R = A; Q = Q'; x = SST(R,b);
endfunction
```

Scanned with CamScanner

Eliminare Gaussiană

$$t_{ip} = \frac{a_{ip}}{a_{pp}}, \quad i = p+1 : n$$

$$t_p = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ t_{p+1,p} \\ \vdots \\ t_{np} \end{bmatrix}$$

$$T_p = I_n - t_p \cdot e_p^T \qquad e_p = p \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

→ sup. Δ

$R = T \cdot A$, unde $T = T_{n-1} \cdot \ldots \cdot T_p \cdot \ldots \cdot T_1$

$A_{p+1} = T_p \cdot A_p$ → inferior Δ

$b_{p+1} = T_p \cdot b_p$

$X = SST(R, Tb)$

complexitate: $O(n^3)$

→ instabil

```
function [x] = G(A,b)
    [n n] = size(A);
    x = zeros(n,1);
    for p = 1: n-1
      for i = p+1 : n
        tp = A(i,p) / A(p,p);
        A(i, p) = 0;
        A(i, p+1 : n) = A(i, p+1 : n) -
                        - A(p, p+1 : n) * tp;
        b(i) = b(i) - b(p) * tp;
      endfor
    endfor
    % SST :
    for i = n:-1:1
    x(i) = (b(i) - A(i, i+1:n) * x(i+1:n))
           / A(i,i);
    endfor
endfunction
```

Scanned with CamScanner

GPPS

La fiecare pas $p = 1 : n-1$, pivotul se obtine astfel:

$$|a_{l_p p}| = \max\{|a_{ip}|\}, \quad i = p : n$$

l_p – linie pivot

pivot

- calcul pivot
- interschimbare $L_1 \leftrightarrow$ Linie pivot
- eliminare gaussiană
- calcul pivot
- $L_2 \leftrightarrow$ Linie pivot
- el. g.

Gand

pivot scalat:

$$\left|\frac{a_{l_p p}}{s_{l_p}}\right| = \max_{i=p:n}\left\{\left|\frac{a_{ip}}{s_i}\right|\right\}$$

$$s_i = \max_{j=p:n}\{|a_{ij}|\}$$

```
function [A b] = GPP(A,b)
    [n n] = size(A);
    for p = 1:n-1
        s = max(abs(A'))';
        pivot = -inf;          % -∞
        linie_pivot = -1;
        % calculez max dintre elem A(p:n,p)
        for j = p:n                    /s(j);
            if pivot < abs(A(j,p))
                pivot = abs(A(j,p))    /s(j);
                linie_pivot = j;
            endif
        endfor
        % permutare linii linie_pivot <-> p
        if p ~= linie_pivot
            for j = p:n
                aux = A(p,j);
                A(p,j) = A(linie_pivot,j);
                A(linie_pivot,j) = t v
            endfor
            aux = b(linie_pivot); b(l p) = b(p); b(p) = aux;
        endif
        % eliminare gaussiana
        for i = p+1:n
            tp = A(i,p)/A(p,p);
            A(i,p) = 0;
            for j = p+1:n;
                A(i,j) = A(1,j) - tp*A(p,j);
            endfor
            b(i) = b(i) - tp*bp; endfor; endfor; endfunction
```

Scanned with CamScanner

GPT

$$|a_{l_p c_p}| = \max_{\substack{p \le i \le n \\ p \le j \le n}} \{ |a_{ij}| \}$$

→ se permută și coloanele, nu doar elimină, "pivotul ajunge în celula"

Scanned with CamScanner

```
function [A,b] = GPT(A,b)
    [n n] = size(A);
    for p = 1:n-1
    % pivot = -inf;
      linie_pivot = -1;
      coloana_pivot = -1;
      % calc. max. m submat. A(p:n, p:n)
      for i = p:n
        for j = p:n
          if pivot < abs(A(i,j));
            pivot = abs(A(i,j));
            linie_pivot = i;
            coloana_pivot = j;
          endif
        endfor
      endfor
      if p ~= linie_pivot
        % linie_pivot <-> p
        % b(liu-piv) <-> b(p)
      endif
      if p ~= coloana_pivot
        % col_piv <-> p
      endif
      % eliminare gaussiana:
      for i = p+1 : n
        if A(p,p) == 0
          continue;
        endif
        tp = A(i,p) / A(p,p);
        A(i,p) = 0;
        for j = p+1 : n
          A(i,j) = A(i,j) - tp * A(p,j);
        endfor
        b(i) = b(i) - tp * b(p);
      endfor
    endfor
endfunction
```

Scanned with CamScanner

Thomas

A matrice tridiagonală

```
b c
a b c
  a b c
    a
```

Complexitate: $O(n)$

$a_i \to a$

$b_i \to b$

$$c'_i = \begin{cases} \frac{c_i}{b_i}, & i = 1 \\ \frac{c_i}{b_i - c'_{i-1} \cdot a_i}, & i = 2:n-1 \end{cases}$$

$$d'_i = \begin{cases} \frac{d_i}{b_i}, & i = 1 \\ \frac{d_i - d'_{i-1} \cdot a_i}{b_i - c'_{i-1} a_i}, & i = 2:n \end{cases}$$

$$\begin{cases} x_n = d'_n \\ x_i = d'_i - c'_i x_{i+1}, & i = n-1:-1:1 \end{cases}$$

Condiție stabilitate: $|b_i| > |a_i| + |c_i|$

Scanned with CamScanner

```
function x = Thomas (a, b, c, d)
    n = length (d);
    % transf. a a. î. să aibă n elem.
    a = [0; a];
    c(1) = c(1)/b(1);
    d(1) = d(1)/b(1);
    for i = 2: n-1
      aux = b(i) - a(i) * c(i-1);
      c(i) = c(i)/aux;
      d(i) = (d(i) - a(i) * d(i-1))/aux;
    endfor
    d(n) = (d(n) - a(n) * d(n-1))/
           /(b(n) - a(n) * c(n-1));
    x(n) = d(n);
    for i = n-1 : -1 : 1
       x(i) = d(i) - c(i) * x(i+1);
    endfor
    x = x';
endfunction
```

Gauss - Sisteme multiple

$$\begin{cases} Ax_1 = b_1 \\ Ax_2 = b_2 \\ \vdots \\ Ax_n = b_n \end{cases} \quad O(n^3)$$

$$\Rightarrow AX = B$$

$$\tilde{B} = T\tilde{B} = [\tilde{b}_1 \ \tilde{b}_2 \ \dots \ \tilde{b}_n] \xrightarrow{\text{inj } \Delta} \tilde{n}$$

$R = TA \rightarrow$ sup Δ

Scanned with CamScanner

```
function X = G_sist_multiple (A, B)
    [n n] = size (A);
    X = zeros (n, n);
    for p = 1 : n-1
      for i = p+1 : n
          tp = A(i, p) / A(p, p);
          A(i, p) = 0;
          A(i, p+1 : n) = A(i, p+1 : n) -
                        - A(p, p+1 : n) * tp;
          B(i, :) = B(i, :) - B(p, :) * tp;
      endfor
    endfor
    for i = n : -1 : 1
      X(i, :) = (B(i, :) - A(i, i+1 : n) *
                 * X(i+1 : n, :)) / A(i, i);
    endfor
endfunction
```

Scanned with CamScanner

Jacobi

$A = D - L - U$

$N = D$

$P = L + U$

$$x_i^{(p+1)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(p)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(p)}}{a_{ii}}$$

$i = 1:n$

$p = 0 : maxiter$

complexitate: $O(n^2 * maxiter)$

- $x^{(p+1)}$ pe diag. prin.
  $x^{(p)}$ pe rest

$$G_J = D^{-1}(L+U)$$

cond: $\rho(G_J) < 1$

```
function [x p] = Jacobi(A, b, x0, tol, maxiter)
    n = length(b);
    x = zeros(n, 1);
    for p = 1 : maxIter
      for i = 1 : n
          x(i) = (b(i) - A(i, 1 : i-1) *
                 * x0(1 : i-1) -
                 - A(i, i+1 : n) * x0(i+1 : n))/
                 / A(i, i);
      endfor
      if norm(x - x0) < tol
          return;
      endif
      x0 = x;
    endfor
endfunction
```

Scanned with CamScanner

Gauss - Seidel

↳ converge daca A este diagonal dominanta

$A = D - L - U$

$N = D - L$

$P = U$

$G_{GS} = N^{-1}P = (D-L)^{-1}U$

$\rho(G_{GS}) < 1 \rightarrow \rho(A) =$ max modul val. proprii

$$x_i^{(p+1)} = \frac{b_{ii} - \sum_{j=1}^{i-1} a_{ij} x_j^{(p+1)} - \sum_{j=i+1}^{n} a_{ij} \cdot x_j^{(p)}}{a_{ii}}$$

$i = 1:n$

$p = 1:$ maxiter

- ce e pe diag. princ. nu depinde de $(p+1)$

Supraralaxare!

$$x_{SOR}^{(p+1)} = \omega \cdot x_{GS}^{(p+1)} + (1-\omega) \cdot x_{SOR}^{(p)}$$

$\omega \in (0,2)$ dată de intrare

$$\omega_{optim} = 1 + \left(\frac{\mu}{1+\sqrt{1-\mu^2}} \right)^2$$

$\mu = \rho(A)$ = valoare proprie maximă (în modul)

```
function [x  p] = GS (A, b, x0, tol, maxiter, w)
    n = length(b);
    x = zeros(n, 1);
    for p = 1 : maxiter
        for i = 1 : n
            x(i) = (b(i) - A(i, 1 : i-1) *
                    * x(1 : i-1) -
                    - A(i, i+1 : n) * x0(i+1 : n))/
                    /A(i, i);
        endfor
        x = w*x + (1-w)*x0;
        if norm (x - x0) < tol
            return;
        endif
        x0 = x;
    endfor
endfunction
```

Scanned with CamScanner

Metoda bisecției

$f:[a,b]\to\mathbb{R}$ cont.

$f(a)\cdot f(b)<0$

$[a,b]\ni$ o singură rad. reală a ec.

x^* sol exactă

$$x^{(1)}=a^{(1)}+\frac{b^{(1)}-a^{(1)}}{2}$$

dacă $f(x^{(1)})<\varepsilon$ sau $|b^{(1)}-a^{(1)}|<\varepsilon$:

aproximația rădăcinii este $x^{(1)}$

dacă $f(x^{(1)})\cdot f(a^{(1)})<0$:

$$a^{(2)}=a^{(1)}$$
$$b^{(2)}=x^{(1)}$$

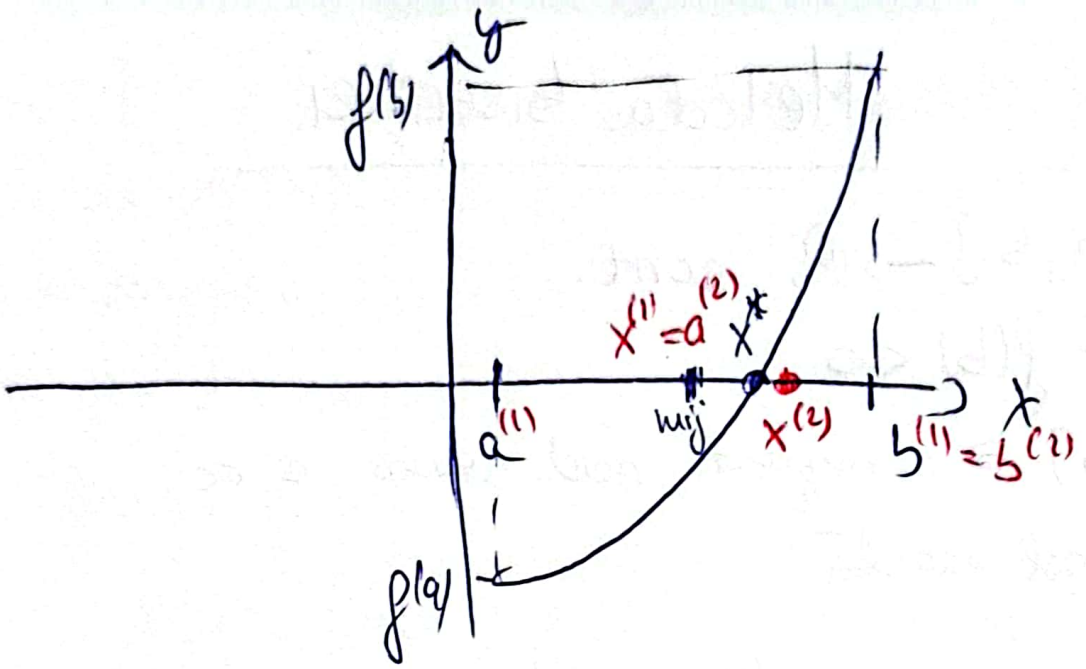
altfel:

$$a^{(2)}=x^{(1)}$$
$$b^{(2)}=b^{(1)}$$

$$x^{(2)}=a^{(2)}+\frac{b^{(2)}-a^{(2)}}{2}$$ samd

$[a^{(4)},b^{(4)}]$: $x^{(4)}=a^{(4)}+\frac{b^{(4)}-a^{(4)}}{2}$

Scanned with CamScanner


y
f(b)
x(1) = a(2)
x*
a(1)
xij
x(2)
b(1) = b(2)
x
f(a)

Metoda secantei

$f:[a,b]\to\mathbb{R}$ cont
$f(a)\cdot f(b)<0$
$[a,b]\ni \alpha$ sg. sol. a ec.
x^* sol exacta

Ec. secantei care trece prin $(a,f(a)), (b,f(b))$:

$$\frac{y-f(a)}{f(b)-f(a)} = \frac{x-a}{b-a}$$

$$\Rightarrow y = f(a) + \frac{x-a}{b-a}(f(b)-f(a))$$

~~Def~~ $\cap Ox$:

$$0 = f(a) + \frac{x^{(1)}-a}{b-a}(f(b)-f(a))$$

$$\Rightarrow x^{(1)} = \frac{a(f(b)-f(a)) - f(a)(b-a)}{f(b)-f(a)}$$

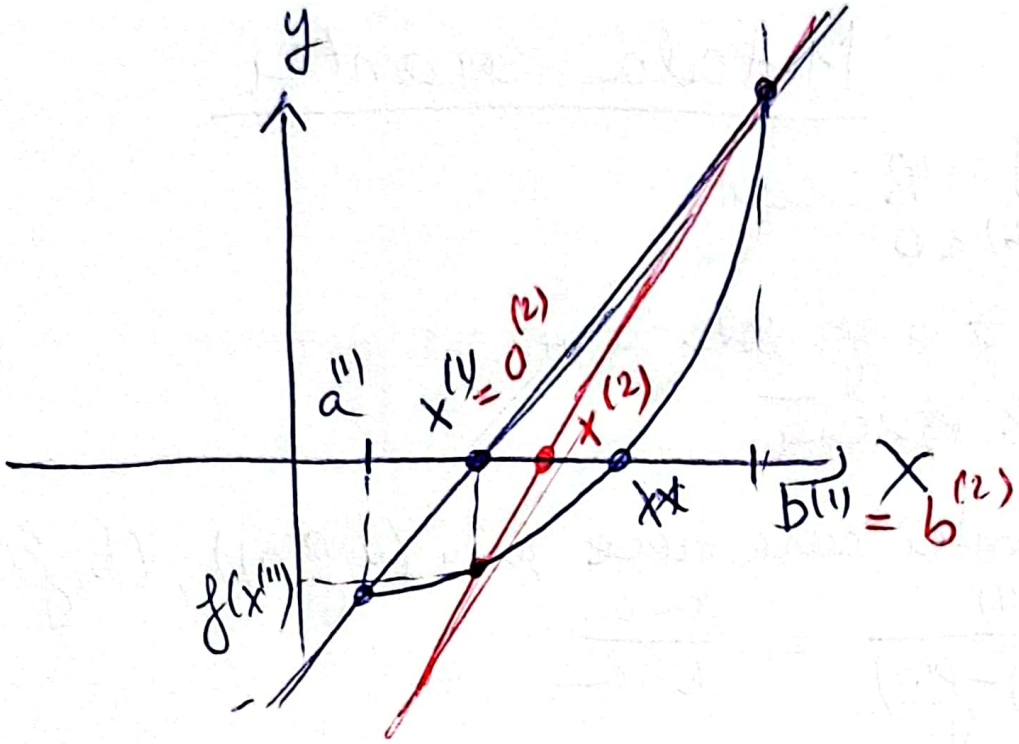
$$x^{(1)} = \frac{a^{(1)}f(b^{(1)}) - b^{(1)}f(a^{(1)})}{f(b^{(1)}) - f(a^{(1)})}$$

if $f(x^{(1)}) < \varepsilon$ sau $|b^{(1)} - a^{(1)}| < \varepsilon$:
aproximatia e $x^{(1)}$

if $f(x^{(1)})\cdot f(a^{(1)}) < 0$ | altfel: $a^{(2)} = x^{(1)}$
$a^{(2)} = a^{(1)}$ | $b^{(2)} = b^{(1)}$
$b^{(2)} = x^{(1)}$

$$x^{(4)} = \frac{a^{(4)}\cdot f(b^{(4)}) - b^{(4)}\cdot f(a^{(4)})}{f(b^{(4)}) - f(a^{(4)})}$$

Scanned with CamScanner



Scanned with CamScanner

Metoda tangentei

Ec. tg:

$$f(x) = f(x^{(n)}) + (x - x^{(n)}) f'(x^{(n)})$$

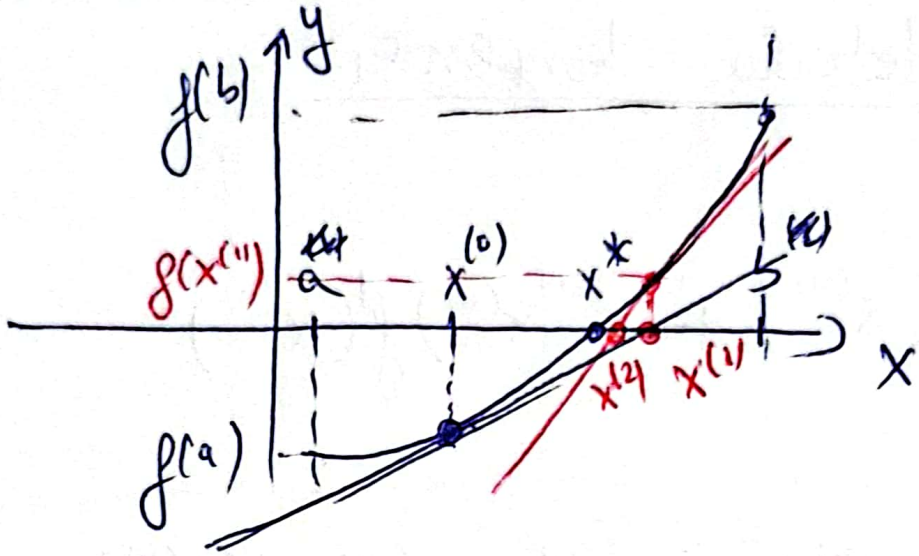
$\cap Ox$:

$$0 = f(x^{(n)}) + (x^{(n+1)} - x^{(n)}) f'(x^{(n)})$$

$$\Rightarrow x^{(n+1)} = x^{(n)} - \frac{f(x^{(n)})}{f'(x^{(n)})}, \quad n \geq 0$$

Date de intrare: $x^{(0)}$, maxiter, tol, functia?

Scanned with CamScanner


y
f(b)
f(x^(1))
a
x^(0)
x*
b
x
x^(2)
x^(1)
f(a)

Sisteme neliniare – met. Newton-Raphson

$$\begin{cases} f_1(x_1,\dots,x_n)=0 \\ \vdots \\ f_n(x_1,\dots,x_n)=0 \end{cases} \quad \left| \quad X=\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}, F(X)=\begin{bmatrix} f_1(x) \\ \vdots \\ f_n(x) \end{bmatrix} \right.$$

$$F(X) = F(x^{(k)}) + J(x^{(k)})(X - x^{(k)})$$

$$J(x)=\begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}$$

$\cap Ox \ni x = x^*$:

$$0 = F(x^{(k)}) + J(x^{(k)})(x^{(k+1)} - x^{(k)})$$

$$\Rightarrow x^{(k+1)} = x^{(k)} - J^{-1}(x^{(k)})F(x^{(k)})$$

Not. $\Delta x^{(k)} = x^{(k-1)} - x^{(k)}$

$$\Delta x^{(k)} = -J^{-1}(x^{(k)})F(x^{(k)})$$

Rezolvăm SEL: $J(x^{(k)})\Delta x^{(k)} = -F(x^{(k)}) \Rightarrow$

$$\Rightarrow x^{(k+1)} = \Delta x^{(k)} + x^{(k)}$$

Scanned with CamScanner